04 – Chat: upravit, smazat, odpovědět, reagovat

1. Co bylo uděláno

Každá zpráva v chatu je teď vlastní řádek přes celou šířku. Když na řádek najedu myší (na mobilu ťuknu), objeví se vedle bubliny lišta s ikonami:

Ikona	Akce	U koho
🖍️ tužka (`Pencil`)	upravit	jen moje zprávy
🗑️ koš (`Trash2`)	smazat	jen moje zprávy
↩️ šipka (`Reply`)	odpovědět	všechny
🙂 smajlík (`Smile`)	reagovat → nabídka 👍 👎 😂 😮	všechny

- **Úprava** i **odpověď** probíhají dole v poli pro psaní. Nad ním se ukáže lišta „Editing message“ / „Replying to …“ s křížkem. Enter uloží/odešle, Esc zruší.
- **Odpověď** má v bublině nahoře citaci původní zprávy. Klik na ni odscrolluje na originál a krátce ho zvýrazní.
- **Reakce**: jeden člověk = jedna reakce na zprávu. Jiné emoji ji vymění, stejné ji odebere. Pod bublinou jsou „čipy“ 👍 2, moje reakce je fialová.
- **Smazání** se potvrzuje dialogem a zpráva zůstane jako „Message deleted“ (soft delete), její reakce zmizí.
- Všechno se druhému ukáže **živě** (Realtime).

2. Nové a změněné soubory

Soubor	K čemu slouží
`supabase/migrations/20261016002_chat_message_actions.sql`	sloupec `reply_to_id`, tabulka `message_reactions`, RPC `set_message_reaction`, triggery, realtime
`apps/web/src/lib/chat-queries.ts`	typy `MessageRow` / `Message` / `Reaction`, `editMessage`, `deleteMessage`, `setReaction`, `fetchMessage`, odpověď v `sendTextMessage`
`apps/web/src/hooks/useMessages.ts`	nové funkce `edit`, `remove`, `react`; realtime i pro UPDATE zpráv a pro reakce
`apps/web/src/components/chat/MessageRow.tsx`	**nový** – jeden řádek zprávy: lišta, výběr emoji, citace odpovědi, čipy reakcí
`apps/web/src/components/chat/ChatThread.tsx`	stav `draft` (odpovídám / upravuji), composer umí oba režimy
`docs/database.md`	popis nové tabulky a sloupce

3. Databáze

Odpověď = jeden sloupec

```
ALTER TABLE messages ADD COLUMN reply_to_id uuid REFERENCES messages(id) ON DELETE SET
NULL;
```

Tabulka odkazuje **sama na sebe** (self-reference). Trigger `tg_messages_before_insert` navíc kontroluje, že zpráva, na kterou odpovídám, je ze **stejné konverzace**. Jinak bych mohl podstrčit cizí `id` z jiného chatu. Klient smí sloupec vyplnit jen při odeslání (`GRANT INSERT (reply_to_id)`), upravit ho později nejde.

Reakce = vlastní tabulka

```
message_reactions (message_id, user_id, conversation_id, emoji, created_at)
PRIMARY KEY (message_id, user_id)   ← jedna reakce na člověka
CHECK (emoji IN ('👍','👎','😂','😮'))
```

- **Proč ne JSON sloupec ve `messages`?** Dva lidé by reagovali současně a jeden by přepsal druhého. A RLS by neuměla hlídat, že měním jen *svou* reakci.
- **Proč je tam `conversation_id`, když jde zjistit přes zprávu?** Denormalizace: RLS politika i realtime filtr `conversation_id=eq.X` pak nepotřebují JOIN.

Zápis jen přes RPC `set_message_reaction(message_id, emoji)`

Funkce v jedné transakci: ověří, že jsem členem a zpráva není smazaná → zjistí moji současnou reakci (`FOR UPDATE` zamkne řádek) → stejné emoji = `DELETE`, jinak `INSERT … ON CONFLICT DO UPDATE` (upsert). Kdyby to dělal klient dvěma dotazy („zjisti stav", pak „zapiš"), dvě rychlá kliknutí by se mohla předběhnout.

Úprava a smazání

Nebylo potřeba nic měnit: policy `messages_update_own` + `GRANT UPDATE (body)` už povolují upravit vlastní text, trigger nastaví `edited_at`. Mazání = existující RPC `delete_message`. Nový trigger `messages_after_soft_delete` (`AFTER UPDATE OF deleted_at … WHEN (OLD.deleted_at IS NULL AND NEW.deleted_at IS NOT NULL)`) smaže reakce smazané zprávy.

4. Jak to funguje krok po kroku

Načtení zpráv – vnořené dotazy (embedding)

```
'…, reactions:message_reactions(user_id, emoji), reply_to:messages!reply_to_id(id,
sender_id, body, deleted_at)'
```

PostgREST podle cizích klíčů (FK) sám přibalí ke každé zprávě pole reakcí a objekt citované zprávy. `!reply_to_id` je **nápověda**, přes který FK se má spojit (tabulka `messages` je tu dvakrát).

Realtime posílá „holý" řádek

Event z Realtime obsahuje jen sloupce tabulky, žádné vnořené `reactions` / `reply_to`. Proto jsou dva typy:

- `MessageRow` = sloupce tabulky (to, co přijde z Realtime),
- `Message = MessageRow + reactions + reply_to` (to, co drží UI).

`fromRow()` doplní prázdné hodnoty. Citace se v UI hledá nejdřív v načtených zprávách (`byId`), takže když autor originál upraví, změní se i citace. Když originál načtený není (je na starší stránce), hook si zprávu dotáhne `fetchMessage()`.

Odběry v `useMessages`

Tabulka	Event	Co se stane
`messages`	INSERT	nová zpráva
`messages`	UPDATE	úprava nebo smazání (`applyRow` – nové sloupce, reakce zůstanou)
`message_reactions`	INSERT / UPDATE	`withReaction(…, emoji)`
`message_reactions`	DELETE	`withReaction(…, null)`

Pozor na DELETE: Supabase u DELETE eventů nepoužije filtr ani RLS (smazaný řádek už nejde zkontrolovat) a při zapnutém RLS pošle jen primární klíč. Proto je odběr DELETE bez filtru. PK je `(message_id, user_id)` , takže víme, co zmizelo, a reakce na zprávu, která v seznamu není, nic nezmění. `REPLICA IDENTITY FULL` říká Postgresu, ať do replikace posílá i starou verzi řádku.

Optimistická reakce

`react()` nejdřív změní stav v UI (hned to vidím), pak zavolá RPC. Když RPC selže, vrátí původní reakce a ukáže chybu. Když pak dorazí realtime „ozvěna" mé vlastní reakce, `withReaction` nastaví tu samou hodnotu, takže se nic nezdvojí (operace je **idempotentní**).

Composer a „draft"

`ChatThread` drží `draft = { kind: 'reply' | 'edit', message } | null` .

- `edit` → composer naplní text zprávy, po Enteru zavolá `edit()` (když se text nezměnil, jen režim zavře),
- `reply` → po Enteru `send(body, replyToId)` .

Text při změně draftu nastavuje composer **přímo během renderu** (`if (draft !== prevDraft) { … setText(…) }`). To je doporučený vzor Reactu pro „stav odvozený ze změny propu". `useEffect` se `setState` by udělal zbytečný render navíc a lint (`react-hooks` v7) ho hlásí.

Hover na celém řádku

Řádek má třídu `group` , lišta `hidden group-hover:flex` . Tailwind tak ukáže lištu, kdykoli je myš kdekoli na řádku. Na dotykových zařízeních (`matchMedia('(hover: none)')`) hover není, proto ťuknutí na řádek nastaví `activeId` a lišta se ukáže natrvalo, dokud nevyberu akci.

5. Nové pojmy

- **Self-reference FK**: sloupec odkazuje na stejnou tabulku (`messages.reply_to_id → messages.id`).
- **Upsert** (`INSERT … ON CONFLICT DO UPDATE`): vlož, a když už řádek existuje, uprav ho.

- `FOR UPDATE`: zamkne vybraný řádek do konce transakce, nikdo jiný ho mezitím nezmění.
- **Denormalizace**: úmyslně uložená kopie dat (`conversation_id`) kvůli rychlosti/jednoduchosti.
- **Replica identity**: co Postgres posílá o starém řádku do replikace (výchozí jen PK, `FULL` celý řádek).
- **Optimistic update**: UI se změní dřív, než server potvrdí; při chybě se vrátí.
- **Idempotence**: provést operaci dvakrát = stejný výsledek jako jednou.

6. Co si mám sám vyzkoušet a ověřit

1. **Spustit migraci** `20261016002_chat_message_actions.sql` (a předtím `20261016001`). Bez ní frontend spadne na načítání zpráv (neexistuje sloupec `reply_to_id`).
2. Dvě okna (A normálně, B anonymně), otevřít stejný chat:
   - A upraví zprávu → B ji uvidí změněnou s „edited" bez obnovení.
   - A smaže zprávu → B uvidí „Message deleted".
   - B dá 👍 → A ji uvidí; B klikne znovu na 👍 → zmizí u obou; B dá 😂 → 👍 se vymění.
   - B odpoví na zprávu A → citace; klik na ni → scroll a zvýraznění.
3. U cizí zprávy se nesmí ukázat tužka ani koš.
4. V SQL editoru zkusit jako uživatel C (není v konverzaci) `select public.set_message_reaction('MSG_ID', '👍');` → čekám chybu `message_not_found` (postup přepnutí na uživatele je v `supabase/tests/chat_rls_checks.sql`).
5. Zkusit `insert into message_reactions …` přímo jako `authenticated` → čekám `permission denied`.

7. Kontrolní otázky

1. Proč je reakce v samostatné tabulce, a ne v JSON sloupci u zprávy?
2. Co zajišťuje primární klíč `(message_id, user_id)`?
3. Proč přepínání reakce dělá RPC, a ne klient dvěma dotazy?
4. Proč odběr DELETE na `message_reactions` nemá filtr podle konverzace?
5. Jaký je rozdíl mezi `MessageRow` a `Message` a proč jsou potřeba oba?
6. Co by se stalo bez kontroly „odpověď je ze stejné konverzace" v triggeru?
7. Proč composer nastavuje text během renderu, a ne v `useEffect`?